Importing the dataset with Data Augmentation

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import warnings

# Suppress all warnings
warnings.filterwarnings("ignore")

# Define the paths
train_dir = '/kaggle/input/alzheimers-data/alzheimer dataset/train'
test_dir = '/kaggle/input/alzheimers-data/alzheimer dataset/test'

# Create an ImageDataGenerator object for data augmentation and
preprocessing
train_datagen = ImageDataGenerator(
    rescale=1./255,
    shear_range=0.4,
    zoom_range=0.2,
    width_shift_range=0.2,
    height_shift_range=0.2,
    rotation_range=90,
    horizontal_flip=True,
    fill_mode='nearest'
)

test_datagen = ImageDataGenerator(rescale=1./255)

# Load the images and labels, ensuring grayscale (1 color channel)
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(128, 128),        # Resize images to 128x128
    color_mode='grayscale',        # □ Load as grayscale
    class_mode='categorical',
)

test_generator = test_datagen.flow_from_directory(
    test_dir,
    target_size=(128, 128),
    color_mode='grayscale',        # □ Load as grayscale
    class_mode='categorical',
)

Found 4647 images belonging to 4 classes.
Found 1230 images belonging to 4 classes.
```

Model Building

CNN MODEL

```
from keras.models import Sequential
from keras.layers import Conv2D
from keras.layers import Flatten
from keras.layers import MaxPooling2D
from keras.layers import Dense
import numpy as np

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten,
Dense, Dropout

# Define the CNN model
CNN1 = Sequential()

# Convolutional layers
CNN1.add(Conv2D(256, (3, 3), activation='relu', input_shape=(128,
128,1
                                                                 )))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))

CNN1.add(Conv2D(64, (3, 3), activation='relu'))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))

CNN1.add(Conv2D(256, (3, 3), activation='relu'))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))

CNN1.add(Conv2D(128, (3, 3), activation='relu'))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))

CNN1.add(Conv2D(256, (3, 3), activation='relu'))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))

# Flattening the layers before feeding into dense layers
CNN1.add(Flatten())

# Fully connected layers
CNN1.add(Dense(128, activation='relu'))
CNN1.add(Dropout(0.2))  # Dropout to prevent overfitting
CNN1.add(Dense(64, activation='relu'))

# Output layer (adjust number of units to match the number of classes)
CNN1.add(Dense(4, activation='softmax'))  # Assuming 4 classes based
on the dataset
```

```
# Compile the model
CNN1.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

# Model summary
CNN1.summary()
```

```
Model: "sequential_14"

┏━━━
```

```
──────┤
│ max_pooling2d_74 (MaxPooling2D) │ (None, 2, 2, 256)      │
0 │
├─────────────────────────────────┼────────────────────────┼──────────
──────┤
│ flatten_14 (Flatten)            │ (None, 1024)           │
0 │
├─────────────────────────────────┼────────────────────────┼──────────
──────┤
│ dense_42 (Dense)                │ (None, 128)            │
131,200 │
├─────────────────────────────────┼────────────────────────┼──────────
──────┤
│ dropout_15 (Dropout)            │ (None, 128)            │
0 │
├─────────────────────────────────┼────────────────────────┼──────────
──────┤
│ dense_43 (Dense)                │ (None, 64)             │
8,256 │
├─────────────────────────────────┼────────────────────────┼──────────
──────┤
│ dense_44 (Dense)                │ (None, 4)              │
260 │
└─────────────────────────────────┴────────────────────────┴──────────
──────┘

 Total params: 1,027,716 (3.92 MB)

 Trainable params: 1,027,716 (3.92 MB)

 Non-trainable params: 0 (0.00 B)
```

```
CNN1.fit(train_generator,epochs=50)
```

```
Epoch 1/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 24s 137ms/step - accuracy: 0.5525 - loss:
1.0294
Epoch 2/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 71ms/step - accuracy: 0.6037 - loss:
0.8612
Epoch 3/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 71ms/step - accuracy: 0.5978 - loss:
0.8627
Epoch 4/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 71ms/step - accuracy: 0.6207 - loss:
0.8075
Epoch 5/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 71ms/step - accuracy: 0.6411 - loss:
0.7838
Epoch 6/50
```

```
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 69ms/step - accuracy: 0.6402 - loss:
0.7639
Epoch 7/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 71ms/step - accuracy: 0.6766 - loss:
0.7313
Epoch 8/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 70ms/step - accuracy: 0.6749 - loss:
0.7161
Epoch 9/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 71ms/step - accuracy: 0.6806 - loss:
0.6966
Epoch 10/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 71ms/step - accuracy: 0.7002 - loss:
0.6605
Epoch 11/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 70ms/step - accuracy: 0.7082 - loss:
0.6549
Epoch 12/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.7014 - loss:
0.6290
Epoch 13/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 70ms/step - accuracy: 0.7375 - loss:
0.5923
Epoch 14/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 69ms/step - accuracy: 0.7315 - loss:
0.5853
Epoch 15/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 71ms/step - accuracy: 0.7415 - loss:
0.5606
Epoch 16/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.7464 - loss:
0.5488
Epoch 17/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 72ms/step - accuracy: 0.7666 - loss:
0.5320
Epoch 18/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 71ms/step - accuracy: 0.7876 - loss:
0.4789
Epoch 19/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.7870 - loss:
0.4698
Epoch 20/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 71ms/step - accuracy: 0.7913 - loss:
0.4612
Epoch 21/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.8083 - loss:
0.4407
Epoch 22/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 70ms/step - accuracy: 0.8261 - loss:
```

```
0.3988
Epoch 23/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 70ms/step - accuracy: 0.8321 - loss:
0.3971
Epoch 24/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 70ms/step - accuracy: 0.8372 - loss:
0.3828
Epoch 25/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.8435 - loss:
0.3674
Epoch 26/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 77ms/step - accuracy: 0.8520 - loss:
0.3468
Epoch 27/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.8602 - loss:
0.3458
Epoch 28/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.8645 - loss:
0.3234
Epoch 29/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.8757 - loss:
0.3088
Epoch 30/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 71ms/step - accuracy: 0.8862 - loss:
0.2791
Epoch 31/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.8809 - loss:
0.2896
Epoch 32/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 77ms/step - accuracy: 0.8900 - loss:
0.2834
Epoch 33/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.9084 - loss:
0.2372
Epoch 34/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.8935 - loss:
0.2596
Epoch 35/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 77ms/step - accuracy: 0.8967 - loss:
0.2572
Epoch 36/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.8948 - loss:
0.2444
Epoch 37/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 71ms/step - accuracy: 0.9049 - loss:
0.2360
Epoch 38/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.9150 - loss:
0.2204
```

```
Epoch 39/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 71ms/step - accuracy: 0.9097 - loss:
0.2236
Epoch 40/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 72ms/step - accuracy: 0.9174 - loss:
0.2106
Epoch 41/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.9302 - loss:
0.1886
Epoch 42/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.9194 - loss:
0.2001
Epoch 43/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.9459 - loss:
0.1550
Epoch 44/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 72ms/step - accuracy: 0.9271 - loss:
0.2047
Epoch 45/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 71ms/step - accuracy: 0.9367 - loss:
0.1644
Epoch 46/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 70ms/step - accuracy: 0.9459 - loss:
0.1421
Epoch 47/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 70ms/step - accuracy: 0.9294 - loss:
0.1831
Epoch 48/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 70ms/step - accuracy: 0.9388 - loss:
0.1567
Epoch 49/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 10s 71ms/step - accuracy: 0.9517 - loss:
0.1334
Epoch 50/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.9419 - loss:
0.1490
```

```
<keras.src.callbacks.history.History at 0x7eee381311d0>
```

```
# Evaluate the model on the test data
loss, accuracy1 = CNN1.evaluate(test_generator)

# Print the accuracy
print(f"Test Accuracy of CNN1: {accuracy1}")
```

```
39/39 ━━━━━━━━━━━━━━━━━━━━ 6s 136ms/step - accuracy: 0.9536 - loss:
0.1373
Test Accuracy of CNN1: 0.9504064917564392
```

#With Batch Normalization

```python
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten,
Dense, Dropout, BatchNormalization

# Define the CNN model
CNN1 = Sequential()

# Convolutional Block 1
CNN1.add(Conv2D(256, (3, 3), activation='relu', input_shape=(128, 128,
1)))
CNN1.add(BatchNormalization())   # Added Batch Normalization
CNN1.add(MaxPooling2D(pool_size=(2, 2)))

# Convolutional Block 2
CNN1.add(Conv2D(64, (3, 3), activation='relu'))
CNN1.add(BatchNormalization())   # Added Batch Normalization
CNN1.add(MaxPooling2D(pool_size=(2, 2)))

# Convolutional Block 3
CNN1.add(Conv2D(256, (3, 3), activation='relu'))
CNN1.add(BatchNormalization())   # Added Batch Normalization
CNN1.add(MaxPooling2D(pool_size=(2, 2)))

# Convolutional Block 4
CNN1.add(Conv2D(128, (3, 3), activation='relu'))
CNN1.add(BatchNormalization())   # Added Batch Normalization
CNN1.add(MaxPooling2D(pool_size=(2, 2)))

# Convolutional Block 5
CNN1.add(Conv2D(256, (3, 3), activation='relu'))
CNN1.add(BatchNormalization())   # Added Batch Normalization
CNN1.add(MaxPooling2D(pool_size=(2, 2)))

# Flatten before Dense layers
CNN1.add(Flatten())

# Fully Connected Layers
CNN1.add(Dense(128, activation='relu'))
CNN1.add(BatchNormalization())   # Optional BN before dropout
CNN1.add(Dropout(0.2))

CNN1.add(Dense(64, activation='relu'))
CNN1.add(BatchNormalization())   # Optional BN before output layer

# Output Layer
CNN1.add(Dense(4, activation='softmax'))  # Assuming 4 classes

# Compile the model
CNN1.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
```

```
# Model summary
CNN1.summary()
```

```
Model: "sequential_15"
```

```
┏━━━
```

```
│ max_pooling2d_77 (MaxPooling2D) │ (None, 14, 14, 256)    │
0 │
├─────────────────────────────────┼────────────────────────┼──────────
──────┤
│ conv2d_78 (Conv2D)              │ (None, 12, 12, 128)    │
295,040 │
├─────────────────────────────────┼────────────────────────┼──────────
──────┤
│ batch_normalization_31          │ (None, 12, 12, 128)    │
512 │
│ (BatchNormalization)            │                        │
│
├─────────────────────────────────┼────────────────────────┼──────────
──────┤
│ max_pooling2d_78 (MaxPooling2D) │ (None, 6, 6, 128)      │
0 │
├─────────────────────────────────┼────────────────────────┼──────────
──────┤
│ conv2d_79 (Conv2D)              │ (None, 4, 4, 256)      │
295,168 │
├─────────────────────────────────┼────────────────────────┼──────────
──────┤
│ batch_normalization_32          │ (None, 4, 4, 256)      │
1,024 │
│ (BatchNormalization)            │                        │
│
├─────────────────────────────────┼────────────────────────┼──────────
──────┤
│ max_pooling2d_79 (MaxPooling2D) │ (None, 2, 2, 256)      │
0 │
├─────────────────────────────────┼────────────────────────┼──────────
──────┤
│ flatten_15 (Flatten)            │ (None, 1024)           │
0 │
├─────────────────────────────────┼────────────────────────┼──────────
──────┤
│ dense_45 (Dense)                │ (None, 128)            │
131,200 │
├─────────────────────────────────┼────────────────────────┼──────────
──────┤
│ batch_normalization_33          │ (None, 128)            │
512 │
│ (BatchNormalization)            │                        │
│
├─────────────────────────────────┼────────────────────────┼──────────
──────┤
│ dropout_16 (Dropout)            │ (None, 128)            │
0 │
├─────────────────────────────────┼────────────────────────┼──────────
```

```
──────┤
│ dense_46 (Dense)                 │ (None, 64)              │
8,256 │
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
│ batch_normalization_34           │ (None, 64)              │
256 │
│ (BatchNormalization)             │                         │
│
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
│ dense_47 (Dense)                 │ (None, 4)               │
260 │
└──────────────────────────────────┴─────────────────────────┴──────────
──────┘

 Total params: 1,032,324 (3.94 MB)

 Trainable params: 1,030,020 (3.93 MB)

 Non-trainable params: 2,304 (9.00 KB)
```

```
CNN1.fit(train_generator,epochs=50)
```

```
Epoch 1/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 22s 100ms/step - accuracy: 0.4878 - loss:
1.3803
Epoch 2/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.6181 - loss:
0.8830
Epoch 3/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.6350 - loss:
0.8247
Epoch 4/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.6561 - loss:
0.7603
Epoch 5/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.6513 - loss:
0.7637
Epoch 6/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.6765 - loss:
0.7120
Epoch 7/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.6850 - loss:
0.7053
Epoch 8/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.6791 - loss:
0.6855
Epoch 9/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.7030 - loss:
```

```
0.6429
Epoch 10/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.7147 - loss:
0.6186
Epoch 11/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.7348 - loss:
0.6198
Epoch 12/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.7547 - loss:
0.5653
Epoch 13/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.7581 - loss:
0.5633
Epoch 14/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 77ms/step - accuracy: 0.7772 - loss:
0.5179
Epoch 15/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 12s 78ms/step - accuracy: 0.7903 - loss:
0.5139
Epoch 16/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 78ms/step - accuracy: 0.8093 - loss:
0.4645
Epoch 17/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.8204 - loss:
0.4386
Epoch 18/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 12s 79ms/step - accuracy: 0.8192 - loss:
0.4309
Epoch 19/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.8423 - loss:
0.3835
Epoch 20/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 77ms/step - accuracy: 0.8485 - loss:
0.3776
Epoch 21/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 77ms/step - accuracy: 0.8582 - loss:
0.3427
Epoch 22/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.8721 - loss:
0.3288
Epoch 23/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 77ms/step - accuracy: 0.8717 - loss:
0.3136
Epoch 24/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.8875 - loss:
0.2872
Epoch 25/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.8956 - loss:
0.2798
```

```
Epoch 26/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.8969 - loss:
0.2774
Epoch 27/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 77ms/step - accuracy: 0.9298 - loss:
0.1991
Epoch 28/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.9170 - loss:
0.2143
Epoch 29/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.9213 - loss:
0.1993
Epoch 30/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 77ms/step - accuracy: 0.9340 - loss:
0.1765
Epoch 31/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.9296 - loss:
0.1807
Epoch 32/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.9390 - loss:
0.1654
Epoch 33/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.9339 - loss:
0.1823
Epoch 34/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.9510 - loss:
0.1423
Epoch 35/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.9497 - loss:
0.1390
Epoch 36/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 12s 79ms/step - accuracy: 0.9405 - loss:
0.1677
Epoch 37/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.9341 - loss:
0.1761
Epoch 38/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.9557 - loss:
0.1240
Epoch 39/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.9496 - loss:
0.1305
Epoch 40/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.9606 - loss:
0.1079
Epoch 41/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 77ms/step - accuracy: 0.9594 - loss:
0.1183
Epoch 42/50
```

```
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 77ms/step - accuracy: 0.9334 - loss:
0.1797
Epoch 43/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.9622 - loss:
0.1047
Epoch 44/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.9695 - loss:
0.0986
Epoch 45/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.9691 - loss:
0.0844
Epoch 46/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 77ms/step - accuracy: 0.9685 - loss:
0.0954
Epoch 47/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 77ms/step - accuracy: 0.9689 - loss:
0.0785
Epoch 48/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.9537 - loss:
0.1461
Epoch 49/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.9778 - loss:
0.0699
Epoch 50/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.9779 - loss:
0.0646
```

```
<keras.src.callbacks.history.History at 0x7eee38393ad0>
```

```
# Evaluate the model on the test data
loss, accuracy1 = CNN1.evaluate(test_generator)

# Print the accuracy
print(f"Test Accuracy of CNN1 with batch normalization: {accuracy1}")
```

```
39/39 ━━━━━━━━━━━━━━━━━━━━ 3s 67ms/step - accuracy: 0.9555 - loss:
0.1255
Test Accuracy of CNN1 with batch normalization: 0.9504064917564392
```

#With L1 regularization

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten,
Dense, Dropout
from tensorflow.keras import regularizers

# Define the CNN model
CNN1 = Sequential()

# Convolutional layers with L1 regularization
```

```
CNN1.add(Conv2D(256, (3, 3), activation='relu', input_shape=(128, 128,
1),
                kernel_regularizer=regularizers.l1(0.001)))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))

CNN1.add(Conv2D(64, (3, 3), activation='relu',
                kernel_regularizer=regularizers.l1(0.001)))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))

CNN1.add(Conv2D(256, (3, 3), activation='relu',
                kernel_regularizer=regularizers.l1(0.001)))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))

CNN1.add(Conv2D(128, (3, 3), activation='relu',
                kernel_regularizer=regularizers.l1(0.001)))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))

CNN1.add(Conv2D(256, (3, 3), activation='relu',
                kernel_regularizer=regularizers.l1(0.001)))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))

# Flatten before Dense layers
CNN1.add(Flatten())

# Fully connected layers with L1 regularization
CNN1.add(Dense(128, activation='relu',
               kernel_regularizer=regularizers.l1(0.001)))
CNN1.add(Dropout(0.2))

CNN1.add(Dense(64, activation='relu',
               kernel_regularizer=regularizers.l1(0.001)))

# Output layer (adjust number of units to match the number of classes)
CNN1.add(Dense(4, activation='softmax'))

# Compile the model
CNN1.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])

# Model summary
CNN1.summary()
```

```
Model: "sequential_10"
```

```
┏━━━
```

```
2,560 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ max_pooling2d_50 (MaxPooling2D) │ (None, 63, 63, 256)    │
0 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ conv2d_51 (Conv2D)               │ (None, 61, 61, 64)     │
147,520 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ max_pooling2d_51 (MaxPooling2D) │ (None, 30, 30, 64)     │
0 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ conv2d_52 (Conv2D)               │ (None, 28, 28, 256)    │
147,712 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ max_pooling2d_52 (MaxPooling2D) │ (None, 14, 14, 256)    │
0 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ conv2d_53 (Conv2D)               │ (None, 12, 12, 128)    │
295,040 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ max_pooling2d_53 (MaxPooling2D) │ (None, 6, 6, 128)      │
0 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ conv2d_54 (Conv2D)               │ (None, 4, 4, 256)      │
295,168 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ max_pooling2d_54 (MaxPooling2D) │ (None, 2, 2, 256)      │
0 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ flatten_10 (Flatten)             │ (None, 1024)           │
0 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ dense_30 (Dense)                 │ (None, 128)            │
131,200 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ dropout_10 (Dropout)             │ (None, 128)            │
0 │
```

```
┣━━━
```

```
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.7522 - loss:
0.5657
Epoch 12/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.7756 - loss:
0.5205
Epoch 13/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.7717 - loss:
0.5191
Epoch 14/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.7940 - loss:
0.4821
Epoch 15/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.8124 - loss:
0.4530
Epoch 16/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.8095 - loss:
0.4628
Epoch 17/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.8265 - loss:
0.4279
Epoch 18/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 72ms/step - accuracy: 0.8472 - loss:
0.3829
Epoch 19/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.8458 - loss:
0.3727
Epoch 20/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.8755 - loss:
0.3151
Epoch 21/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.8825 - loss:
0.2984
Epoch 22/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.8943 - loss:
0.2781
Epoch 23/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.9054 - loss:
0.2549
Epoch 24/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.9028 - loss:
0.2480
Epoch 25/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.9144 - loss:
0.2250
Epoch 26/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.9236 - loss:
0.2135
Epoch 27/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.9377 - loss:
```

```
0.1782
Epoch 28/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.9128 - loss:
0.2300
Epoch 29/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.9389 - loss:
0.1760
Epoch 30/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.9408 - loss:
0.1765
Epoch 31/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.9441 - loss:
0.1588
Epoch 32/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.9506 - loss:
0.1358
Epoch 33/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.9236 - loss:
0.2059
Epoch 34/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.9583 - loss:
0.1154
Epoch 35/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.9515 - loss:
0.1312
Epoch 36/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.9563 - loss:
0.1259
Epoch 37/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 12s 78ms/step - accuracy: 0.9554 - loss:
0.1240
Epoch 38/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.9585 - loss:
0.1279
Epoch 39/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.9640 - loss:
0.1099
Epoch 40/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.9689 - loss:
0.0917
Epoch 41/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.9722 - loss:
0.0812
Epoch 42/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.9646 - loss:
0.1044
Epoch 43/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.9639 - loss:
0.1096
```

```
Epoch 44/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.9690 - loss:
0.0864
Epoch 45/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.9730 - loss:
0.0872
Epoch 46/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.9584 - loss:
0.1062
Epoch 47/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.9638 - loss:
0.0901
Epoch 48/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.9770 - loss:
0.0639
Epoch 49/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.9736 - loss:
0.0679
Epoch 50/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.9668 - loss:
0.0890

<keras.src.callbacks.history.History at 0x7eee0edce050>

# Evaluate the model on the test data
loss, accuracy1 = CNN1.evaluate(test_generator)

# Print the accuracy
print(f"Test Accuracy of CNN1 with L1 Regularization: {accuracy1}")

39/39 ━━━━━━━━━━━━━━━━━━━━ 3s 64ms/step - accuracy: 0.9867 - loss:
0.0383
Test Accuracy of CNN1 with L1 Regularization: 0.9821138381958008
```

With Batch Normalization, L2 regularization, Dropout

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (Conv2D, MaxPooling2D, Flatten,
Dense, Dropout,
                                     BatchNormalization, Activation,
SpatialDropout2D)
from tensorflow.keras import regularizers
from tensorflow.keras.optimizers import Adam



CNN1 = Sequential()

# ---- Block 1 ----
CNN1.add(Conv2D(256, (3, 3), input_shape=(128, 128, 1),
```

```
                kernel_regularizer=regularizers.l2(0.001)))
CNN1.add(BatchNormalization())
CNN1.add(Activation('relu'))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))
CNN1.add(SpatialDropout2D(0.15))   # small dropout after conv/pool
(optional)

# ---- Block 2 ----
CNN1.add(Conv2D(64, (3, 3),
                kernel_regularizer=regularizers.l2(0.001)))
CNN1.add(BatchNormalization())
CNN1.add(Activation('relu'))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))
CNN1.add(SpatialDropout2D(0.15))

# ---- Block 3 ----
CNN1.add(Conv2D(256, (3, 3),
                kernel_regularizer=regularizers.l2(0.001)))
CNN1.add(BatchNormalization())
CNN1.add(Activation('relu'))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))
CNN1.add(SpatialDropout2D(0.15))

# ---- Block 4 ----
CNN1.add(Conv2D(128, (3, 3),
                kernel_regularizer=regularizers.l2(0.001)))
CNN1.add(BatchNormalization())
CNN1.add(Activation('relu'))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))
CNN1.add(SpatialDropout2D(0.15))

# ---- Block 5 ----
CNN1.add(Conv2D(256, (3, 3),
                kernel_regularizer=regularizers.l2(0.001)))
CNN1.add(BatchNormalization())
CNN1.add(Activation('relu'))
CNN1.add(MaxPooling2D(pool_size=(2, 2)))
CNN1.add(SpatialDropout2D(0.15))

# ---- Classifier ----
CNN1.add(Flatten())

CNN1.add(Dense(128, kernel_regularizer=regularizers.l2(0.001)))
CNN1.add(BatchNormalization())
CNN1.add(Activation('relu'))
CNN1.add(Dropout(0.5))             # higher dropout on dense layers

CNN1.add(Dense(64, kernel_regularizer=regularizers.l2(0.001)))
CNN1.add(BatchNormalization())
CNN1.add(Activation('relu'))
```

```
CNN1.add(Dropout(0.5))

CNN1.add(Dense(4, activation='softmax'))

# Compile with slightly lower LR (pairs well with dropout/BN)
CNN1.compile(optimizer=Adam(learning_rate=0.0004),
             loss='categorical_crossentropy',
             metrics=['accuracy'])

CNN1.summary()
```

```
Model: "sequential_12"

┏━━━
```

```
│
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ activation_1 (Activation)            │ (None, 61, 61, 64)           │
0 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ max_pooling2d_61 (MaxPooling2D)      │ (None, 30, 30, 64)           │
0 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ spatial_dropout2d_1                  │ (None, 30, 30, 64)           │
0 │
│ (SpatialDropout2D)                   │                              │
│
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ conv2d_62 (Conv2D)                   │ (None, 28, 28, 256)          │
147,712 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ batch_normalization_16               │ (None, 28, 28, 256)          │
1,024 │
│ (BatchNormalization)                 │                              │
│
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ activation_2 (Activation)            │ (None, 28, 28, 256)          │
0 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ max_pooling2d_62 (MaxPooling2D)      │ (None, 14, 14, 256)          │
0 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ spatial_dropout2d_2                  │ (None, 14, 14, 256)          │
0 │
│ (SpatialDropout2D)                   │                              │
│
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ conv2d_63 (Conv2D)                   │ (None, 12, 12, 128)          │
295,040 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ batch_normalization_17               │ (None, 12, 12, 128)          │
512 │
│ (BatchNormalization)                 │                              │
│
```

```
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ activation_3 (Activation)            │ (None, 12, 12, 128)          │
0 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ max_pooling2d_63 (MaxPooling2D)      │ (None, 6, 6, 128)            │
0 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ spatial_dropout2d_3                  │ (None, 6, 6, 128)            │
0 │
│ (SpatialDropout2D)                   │                              │
│
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ conv2d_64 (Conv2D)                   │ (None, 4, 4, 256)            │
295,168 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ batch_normalization_18               │ (None, 4, 4, 256)            │
1,024 │
│ (BatchNormalization)                 │                              │
│
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ activation_4 (Activation)            │ (None, 4, 4, 256)            │
0 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ max_pooling2d_64 (MaxPooling2D)      │ (None, 2, 2, 256)            │
0 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ spatial_dropout2d_4                  │ (None, 2, 2, 256)            │
0 │
│ (SpatialDropout2D)                   │                              │
│
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ flatten_12 (Flatten)                 │ (None, 1024)                 │
0 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ dense_36 (Dense)                     │ (None, 128)                  │
131,200 │
├──────────────────────────────────────┼──────────────────────────────┼──────────────
──────┤
│ batch_normalization_19               │ (None, 128)                  │
```

```
512 │
│ (BatchNormalization)             │                         │
│
├──────────────────────────────────┼─────────────────────────┼──────────────
──────┤
│ activation_5 (Activation)        │ (None, 128)             │
0 │
├──────────────────────────────────┼─────────────────────────┼──────────────
──────┤
│ dropout_12 (Dropout)             │ (None, 128)             │
0 │
├──────────────────────────────────┼─────────────────────────┼──────────────
──────┤
│ dense_37 (Dense)                 │ (None, 64)              │
8,256 │
├──────────────────────────────────┼─────────────────────────┼──────────────
──────┤
│ batch_normalization_20           │ (None, 64)              │
256 │
│ (BatchNormalization)             │                         │
│
├──────────────────────────────────┼─────────────────────────┼──────────────
──────┤
│ activation_6 (Activation)        │ (None, 64)              │
0 │
├──────────────────────────────────┼─────────────────────────┼──────────────
──────┤
│ dropout_13 (Dropout)             │ (None, 64)              │
0 │
├──────────────────────────────────┼─────────────────────────┼──────────────
──────┤
│ dense_38 (Dense)                 │ (None, 4)               │
260 │
└──────────────────────────────────┴─────────────────────────┴──────────────
──────┘

 Total params: 1,032,324 (3.94 MB)

 Trainable params: 1,030,020 (3.93 MB)

 Non-trainable params: 2,304 (9.00 KB)
```

```
CNN1.fit(train_generator,epochs=50)
```

```
Epoch 1/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 23s 107ms/step - accuracy: 0.4775 - loss:
1.4152
Epoch 2/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 77ms/step - accuracy: 0.6182 - loss:
0.8906
```

```
Epoch 3/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.6413 - loss:
0.8015
Epoch 4/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 72ms/step - accuracy: 0.6530 - loss:
0.7535
Epoch 5/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.6579 - loss:
0.7280
Epoch 6/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 77ms/step - accuracy: 0.6753 - loss:
0.7039
Epoch 7/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.6889 - loss:
0.7015
Epoch 8/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.7175 - loss:
0.6289
Epoch 9/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.7290 - loss:
0.6167
Epoch 10/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.7318 - loss:
0.5934
Epoch 11/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.7440 - loss:
0.5763
Epoch 12/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.7589 - loss:
0.5548
Epoch 13/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.7810 - loss:
0.5073
Epoch 14/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.7941 - loss:
0.4908
Epoch 15/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.8120 - loss:
0.4572
Epoch 16/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.8298 - loss:
0.4289
Epoch 17/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 72ms/step - accuracy: 0.8358 - loss:
0.4103
Epoch 18/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.8312 - loss:
0.3972
Epoch 19/50
```

```
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.8519 - loss:
0.3697
Epoch 20/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.8716 - loss:
0.3296
Epoch 21/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.8790 - loss:
0.3058
Epoch 22/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 77ms/step - accuracy: 0.9018 - loss:
0.2536
Epoch 23/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.8986 - loss:
0.2566
Epoch 24/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.9091 - loss:
0.2381
Epoch 25/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.9132 - loss:
0.2296
Epoch 26/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.9066 - loss:
0.2358
Epoch 27/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.9360 - loss:
0.1782
Epoch 28/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.9201 - loss:
0.2133
Epoch 29/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.9394 - loss:
0.1678
Epoch 30/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.9523 - loss:
0.1419
Epoch 31/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.9410 - loss:
0.1659
Epoch 32/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.9255 - loss:
0.1940
Epoch 33/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.9510 - loss:
0.1332
Epoch 34/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.9610 - loss:
0.1077
Epoch 35/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.9425 - loss:
```

```
0.1570
Epoch 36/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.9539 - loss:
0.1474
Epoch 37/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.9571 - loss:
0.1249
Epoch 38/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.9558 - loss:
0.1197
Epoch 39/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.9620 - loss:
0.1102
Epoch 40/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.9633 - loss:
0.1005
Epoch 41/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.9710 - loss:
0.0864
Epoch 42/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.9691 - loss:
0.0913
Epoch 43/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.9686 - loss:
0.0889
Epoch 44/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.9700 - loss:
0.0869
Epoch 45/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.9712 - loss:
0.0919
Epoch 46/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 74ms/step - accuracy: 0.9693 - loss:
0.0887
Epoch 47/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.9749 - loss:
0.0714
Epoch 48/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 73ms/step - accuracy: 0.9660 - loss:
0.0944
Epoch 49/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 75ms/step - accuracy: 0.9733 - loss:
0.0722
Epoch 50/50
146/146 ━━━━━━━━━━━━━━━━━━━━ 11s 76ms/step - accuracy: 0.9865 - loss:
0.0452

<keras.src.callbacks.history.History at 0x7eee0cdc3ad0>
```

```python
# Evaluate the model on the test data
loss, accuracy1 = CNN1.evaluate(test_generator)

# Print the accuracy
print(f"Test Accuracy of CNN1 with L2 Regularization: {accuracy1}")
```

```
39/39 ━━━━━━━━━━━━━━━━━━━━ 3s 65ms/step - accuracy: 0.9807 - loss:
0.0797
Test Accuracy of CNN1 with L2 Regularization: 0.9747967720031738
```